

Writing Part

소유자	Hyejun Park
생성 일시	@2025년 12월 4일 오전 4:35
시험언급	☐

2026 PACE Challenge focuses on the **(rooted) Maximum-Agreement Forest** problem.

Receive several **rooted phylogenetic trees**—all on the same set of leaf labels. → **cut** each tree (remove edges) and **clean it up** (contract boring degree-1 nodes) so that *all* the trees reduce to the *same* forest.

Also forest should have the **fewest possible components**.

Fewer components = more “agreement” among all input trees.

3 tracks:

- **Exact:** find the *optimal* smallest forest.
- **Heuristic:** find *good* forests fast (not guaranteed optimal).
- **Lower-bound:** compute how low the optimum could possibly be.

Approach

1. Understand what the problem wants.

I'll study how rooted trees are represented and what exactly makes two trees “disagree.”

To do this, I'll look at clusters or parent–child relationships in each tree and see how they differ.

Understanding this structure helps me see *why* cuts are needed.

2. Build the basic tools.

I'll write code for parsing Newick trees and storing them in a form that is easy to compare (e.g., adjacency lists or simple node objects).

I'll also implement functions for cutting edges and contracting nodes so I can simulate possible solutions.

3. Pick the track I want to target.

The choice affects the whole algorithm style:

- Exact → search + pruning
- Heuristic → greedy + local search
- Lower bound → structural analysis

So I'll pick one early.

4. Make a simple baseline solution.

I'll use one tree as a "reference" and cut edges in the other trees whenever their structure conflicts with the reference.

This is not optimal, but it gives me a correct agreement forest and something to measure against.

5. Detect disagreements in a systematic way.

I'll compare trees using simple techniques like:

- checking which leaf sets (clusters) are present in each tree,
- identifying edges that cannot exist in both trees at the same time,
- or finding conflicting ancestor relationships.

Edges involved in these conflicts become candidates for cutting.

6. Improve the strategy with basic algorithm ideas.

Depending on the track:

- **Exact track:**

I'll try a small search algorithm.

For each conflict, I can branch on which edge to cut, reduce the search space using pruning rules, and stop when the number of cuts exceeds the best known solution.

- **Heuristic track:**

I'll try greedy rules like "cut the edge that appears in the most conflicts" or "prefer cuts high in the tree."

After that, I can run a simple local search: try undoing or swapping cuts and see if the forest becomes smaller.

- **Lower-bound track:**

I'll compute simple lower bounds, like counting unavoidable conflicts, or using the number of incompatible clusters.

These ideas give the approach some real algorithmic structure without being too complicated.

7. Test and refine.

I'll run my solver on sample instances, check how many components I get, and measure how long it takes.

If some parts are slow or produce too many components, I'll adjust the strategy—for example, change how conflicts are detected or add rules for picking better cuts.

8. Final cleanup and submission.

After everything works reliably, I'll clean the code, verify formatting, and prepare the final version.